

BioEval Reproducibility Report

Computational Simulation Papers — Generated June 4, 2026

This report presents structured reproducibility evaluations for 14 computational simulation projects. Each is scored across six dimensions: Data Disclosure, Dataset Resolvability, Code Availability, Traceability, Simulation Clarity, and Reproducibility Package Quality. Papers are ordered by overall score.

All Papers — Score Summary							
Paper	Score	Disclosure	Dataset	Code	Trace	Sim	Repro
Ant Simulation — Pygame Stigmergy	85.4	92	98	83	88	75	72
Firefly Synchronization Simulation (JS...	73.6	82	78	78	75	68	52
BeeStack: An Evidence-Typed Scaffold f...	73.1	72	58	88	75	78	62
The Ant Stack: Reproducible scientific...	72.2	72	68	92	62	66	82
Termite Colony — Unity 3D Multiagent	71.6	82	78	62	75	72	48
Bee Swarm Simulation	68.1	72	68	75	62	58	82
Rust Ant Colony Simulation	64.3	82	75	72	48	52	58
ACO Robot Path Planning (Python)	54.5	52	58	68	62	50	28
Ant Colony Optimization — React/Canvas	52.7	58	62	72	42	38	50
Locas Ants — Lua/Love2D Ant Colony	51.6	48	55	82	38	38	62
Ants Sandbox (TypeScript/web)	51.1	48	58	78	38	40	55
Symbiants — Ant Colony Sim (Rust)	43.8	38	42	78	30	32	58
swarm-abc — Artificial Bee Colony Simu...	41.9	38	48	62	35	40	28
Hardware-Accelerated Ant Colony Swarm ...	37.8	32	38	58	35	30	40

Ant Simulation — Pygame Stigmergy

Source: <https://github.com/lrdcxdes/ant-simulation>

Evaluated: June 3, 2026

85.4

Overall /100



Summary

This is a pure agent-based simulation with no external datasets; reproducibility therefore hinges entirely on the completeness of generative inputs (rules, parameters, initial conditions). The repository is publicly accessible under MIT license with a requirements.txt, a demo video, and named tunable constants in sim.py, which is a reasonable foundation. However, critical gaps remain: Python and dependency versions are unpinned, default parameter values are undocumented in the README, no random seed is specified (making runs non-deterministic), and there is no CI, testing infrastructure, or archived snapshot (e.g., Zenodo DOI). The shallow commit history (7 commits, 0 releases/tags) and absence of a formal publication or DOI further limit reproducibility confidence.

Positive Findings

- Repository is publicly accessible on GitHub under the MIT License, permitting unrestricted reuse and forking
- A requirements.txt dependency manifest is present, listing Pygame, NumPy, and SciPy as dependencies
- A README is present (~3660 bytes) describing the simulation's conceptual framework, core principles, and how to run it
- Named simulation parameters (ANT_COUNT, EVAPORATION_RATE, NEST_GRAVITY_STRENGTH, MAX_PATIENCE, PANIC_DURATION) are exposed as top-level constants in sim.py, facilitating experimentation
- A demo video (demo.mp4) is included in the repository to illustrate expected simulation output
- No external datasets are used; the simulation is entirely self-contained and generative, which is appropriate for this class of model
- Installation and execution instructions are provided as three clear shell commands
- The simulation logic is contained in a single file (sim.py), reducing the complexity of code navigation

Gaps Identified

- Python version and specific library versions (Pygame, NumPy, SciPy) are not pinned in requirements.txt, making exact environment reconstruction uncertain
- Default values for key parameters (ANT_COUNT, EVAPORATION_RATE, etc.) are not documented in the README, so the described behavior cannot be directly reproduced without reading source code
- No random seed is specified or documented; interactive mouse-driven food placement introduces non-determinism, making run-to-run comparison impossible without explicit seeding
- No versioned release or tagged commit exists (0 published releases, 0 tags), preventing citation of a fixed code state
- No CI/CD or automated testing infrastructure is present, so behavioral correctness after dependency updates is unverifiable
- No archived snapshot (e.g., Zenodo DOI) is linked, meaning long-term accessibility depends solely on GitHub availability
- No formal paper or supplementary methods document is linked; the README serves as the sole scientific description with limited quantitative detail
- Simulation grid dimensions, initial ant placement, and pheromone decay functional form are not described in the README

Recommendations

- Pin all dependency versions in requirements.txt (e.g., pygame==2.5.2, numpy==1.26.4, scipy==1.13.0) and specify the tested Python version (e.g., in a .python-version file or pyproject.toml)
- Document all default parameter values and their units/ranges in the README or a dedicated PARAMETERS.md file so that the described behavior is reproducible without inspecting source code
- Add a --seed command-line argument and document a reference seed value that reproduces the behavior shown in demo.mp4, eliminating non-determinism
- Create a tagged release (e.g., v1.0.0) and archive it to Zenodo to obtain a citable DOI and ensure long-term preservation
- Add a minimal test suite (e.g., a headless pytest that runs the simulation for N steps and checks pheromone grid properties) and wire it to a GitHub Actions CI workflow
- Extend the README to specify grid dimensions, initial conditions, and the mathematical form of pheromone evaporation/diffusion, enabling independent reimplementations
- Consider providing a configuration file (e.g., config.yaml) that encodes the reference parameter set, so users can version-control and share experimental configurations

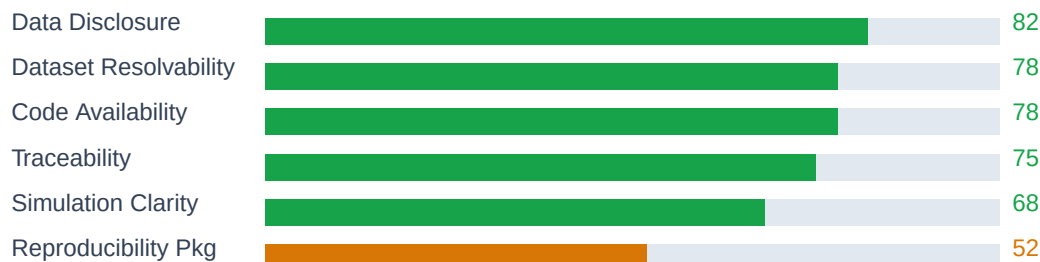
Firefly Synchronization Simulation (JS)

Source: <https://github.com/lax4mike/firefly>

Evaluated: June 3, 2026

73.6

Overall /100



Summary

This repository provides a publicly accessible firefly synchronization simulation with both a JavaScript web application and an Arduino C++ implementation, and notably includes the Mirolo-Strogatz (1990) foundational paper as a PDF alongside a firefly algorithms document — giving at least partial traceability of the theoretical basis. However, the codebase lacks a license, tagged releases, CI/CD, tests, a meaningful README (only ~42 bytes), and any formal archive (e.g., Zenodo DOI), substantially limiting reproducibility. Since this is a pure simulation with no external datasets, scoring for data disclosure and resolvability is redirected to the completeness of generative inputs (parameters, initial conditions, configuration files), which are not explicitly documented outside of the code itself.

Positive Findings

- Source code is publicly accessible on GitHub with 85 commits and active history through 2021
- The Mirolo-Strogatz 1990 foundational paper PDF is included directly in the repository, providing theoretical grounding
- A separate firefly algorithms PDF is included, suggesting algorithmic parameter sources are at least partially referenced
- The simulation covers both a deployed web application (JavaScript) and a physical hardware implementation (Arduino/C++), broadening reproducibility scope
- A package-lock.json dependency manifest is present, partially pinning JavaScript dependency versions
- The simulation is deployed as a live web application, enabling immediate interactive inspection
- Build tooling configuration files (.babelrc, .eslintrc.json, gulpfile.js) are present, documenting the build environment

Gaps Identified

- No software license is detected, making legal reuse and redistribution ambiguous
- README is effectively empty (~42 bytes), providing no documentation of parameters, usage instructions, or algorithm details
- No tagged releases or pinned commits are published, preventing citation of a specific reproducible version
- No CI/CD workflows or automated tests are present, so correctness of the simulation cannot be verified automatically
- No changelog or release notes document how simulation parameters or algorithms have evolved across 85 commits
- Hardware specifications and simulation parameters for the Arduino physical implementation are not documented
- No explicit data availability statement or parameter table is provided; initial conditions and algorithm constants are only recoverable by reading source code
- No formal archive deposit (e.g., Zenodo) exists, leaving the repository without a persistent identifier
- No simulation output files or result datasets are included, preventing independent validation of published results
- [Critic] The README is only ~42 bytes, which is insufficient to document usage instructions, parameter descriptions, or reproduction steps; any paper claims about documentation quality or ease of reproduction should be treated cautiously.
- [Critic] No tests directory and no CI configuration are detected, so correctness of the simulation implementation relative to the Mirolo-Strogatz model cannot be independently verified from repository signals alone.
- [Critic] The repository has 0 published releases and 0 tags, and the last push was 57 months ago; if the paper implies the software is actively maintained or versioned, that would be unsupported by the evidence.

Recommendations

- Add a software license (e.g., MIT or Apache 2.0) to clarify reuse terms
- Expand the README to document: algorithm parameters with references to the included PDFs, initial conditions, hardware requirements for Arduino, and step-by-step build/run instructions
- Create a tagged release (e.g., v1.0.0) pinned to the commit corresponding to any published or presented results
- Deposit the repository to Zenodo to obtain a persistent DOI for citation
- Add a parameter table or configuration file explicitly linking each simulation constant to its source in the Mirolo-Strogatz paper or the algorithms PDF
- Implement at least basic automated tests (e.g., unit tests for the synchronization logic) and configure GitHub Actions CI to run them
- Document the Arduino hardware specifications (model, wiring, timing constraints) to enable physical replication
- Add a CHANGELOG.md summarizing how simulation parameters or algorithms changed across major development phases
- Consider including example simulation output files or screenshots to allow comparison against independently reproduced results

BeeStack: An Evidence-Typed Scaffold for Whole-Colony Honeybee Simulation

Source: <https://zenodo.org/records/20420557>

Evaluated: June 3, 2026

73.1

Overall /100



Summary

BeeStack demonstrates commendable transparency for a simulation scaffold: the Zenodo deposit (DOI 10.5281/zenodo.20420557) provides a versioned source archive with MD5 checksums, and the GitHub repository contains DOI-cited empirical references in source files (docs/beebrain_data_pipeline.md, manuscript/references.bib) pointing to resolvable datasets on figshare, Dryad, and journals. However, the GitHub repository lacks a detected license file (contradicting the MIT claim on Zenodo), has no CI/CD workflow, a minimal README (~259 bytes), and no pinned dependency versions in pyproject.toml, collectively limiting practical reproducibility. One DOI (10.1186/s12864-019-5639-3) returned unresolved, and key dependencies (FlyBody, MuJoCo versions) are not explicitly pinned, leaving environment reconstruction partially manual.

Positive Findings

- Source code is publicly archived on Zenodo (DOI 10.5281/zenodo.20420557) with MD5 checksums for both the tarball and PDF, enabling integrity verification
- A versioned GitHub release (v1.0.0 tag) exists, providing a pinned reference point for the code state
- Repository source files (docs/beebrain_data_pipeline.md, manuscript/references.bib) contain explicit DOIs for empirical datasets including figshare (10.6084/m9.figshare.24715977.v1), Dryad (10.5061/dryad.*), and journal articles, demonstrating genuine code-to-data traceability
- 9 of 10 DOIs found in repository source files resolve successfully to public resources
- The manuscript describes an explicit philosophy of cataloguing evidence gaps rather than hiding them, and declares validation rate as a config-band self-test measure
- A dependency manifest (pyproject.toml) is present, and a tests directory exists in the repository
- The scaffold's five-layer architecture (Body, Brain, Mind, Swarm, Niche) is described with per-module fidelity declarations, and every quoted number is described as traceable from configuration to artifact to manuscript
- Software is released under MIT License (declared on Zenodo deposit)
- BEEHAVE is referenced as a related tool, and multiple neuroanatomical DOIs (antennal lobe, mushroom body connectome data) are cited in beebrain_data_pipeline.md

Gaps Identified

- GitHub repository API signals show no license file detected in the repository itself, creating a discrepancy with the MIT license declared on Zenodo; downstream users cloning the repo directly may not encounter a LICENSE file
- README is minimal (~259 bytes), providing insufficient documentation for environment setup, parameter configuration, or workflow execution without consulting external files
- No CI/CD (GitHub Actions) workflows are detected, meaning automated reproducibility testing is absent and the tests directory contents cannot be confirmed to pass
- pyproject.toml is present but dependency versions are not confirmed to be pinned, and FlyBody and MuJoCo version numbers are not specified in the Zenodo description or pre-extracted evidence
- One DOI (10.1186/s12864-019-5639-3) is unresolved (not found in Crossref or DataCite), leaving one cited dataset inaccessible
- No Dockerfile or containerized environment definition is present, requiring manual environment reconstruction
- Waggle dance observational data and specific electrophysiology/connectome accession numbers for antennal lobe, mushroom body, and central complex models are not cited with accession numbers in the Zenodo description (though some DOIs appear in repo source files)
- No example/test data or one-command execution workflow is documented in accessible form given the minimal README
- [Critic] The GitHub repository README is only ~259 bytes, which is insufficient to document usage, installation, or reproduction steps; combined with the absence of CI workflows and a Dockerfile, independent re-execution of the pipeline is not straightforwardly supported by the repository infrastructure as presented.
- [Critic] The DOI 10.1186/s12864-019-5639-3, found in the repository source file docs/beebrain_data_pipeline.md, could not be resolved in Crossref or DataCite, meaning one cited data/software dependency in the pipeline documentation has an unverified reference.
- [Critic] The accession 10.23719/1523343 found in manuscript/03_materials_and_source_provenance.md was not checked and its resolution status is unknown; it does not follow a standard DOI prefix pattern for common repositories and warrants verification.
- [Critic] Several Dryad DOIs referenced in manuscript/references.bib (e.g., 10.5061/dryad.qbzk18sc, 10.5061/dryad.83bk3j9tt, 10.5061/dryad.rv15dv4k2, 10.5061/dryad.qjq2bvqw6, 10.5061/dryad.rj10c) were not independently checked for resolution in this audit run and their status remains unknown.
- [Critic] The DOI 10.1038/s41586-025-09029-4 found in manuscript/references.bib was not included in the verified or unresolved lists and

its resolution status is unknown.

Recommendations

- Add a LICENSE file directly to the GitHub repository root to ensure MIT license terms are unambiguous for users who clone without visiting Zenodo
 - Expand the README to at minimum 1-2 pages covering: environment setup steps, how to run a minimal simulation, and a pointer to the five-layer configuration system
 - Add a GitHub Actions CI workflow that installs dependencies and runs the test suite, providing automated verification that the scaffold executes correctly
 - Pin all dependency versions in pyproject.toml (or add a requirements.txt/conda environment.yml with locked versions) and explicitly state FlyBody and MuJoCo version numbers in the README and Zenodo description
 - Investigate and resolve or replace DOI 10.1186/s12864-019-5639-3, which currently returns not-found in both Crossref and DataCite
 - Add a Dockerfile or conda environment specification to enable one-command environment reconstruction
 - Include a minimal worked example with bundled test data and expected output checksums so users can verify their installation produces correct results
 - For each brain structure module (antennal lobe, mushroom body, central complex), add an explicit accession-number table in the manuscript or README mapping each modeled parameter to its empirical source DOI and the specific value/table used
-

The Ant Stack: Reproducible scientific workspace for ant-inspired simulation and complexity energetics

Source: —

Evaluated: June 3, 2026

72.2

Overall /100



Summary

The Ant Stack presents a well-structured simulation framework with a public GitHub repository (`ant_stack`), a passing test suite (676 tests), manifest-tracked artifacts with SHA-256 checksums, and CLI-driven workflows — all positive reproducibility signals. However, the paper is primarily a design specification and roadmap rather than a completed empirical study: core components (AntBody, AntBrain, AntMind) are described as targets rather than implemented and validated artifacts, and no external datasets are provided with resolvable accessions despite the paper citing field data pipelines and tools like DeepLabCut and idtracker.ai. Generative input specifications (YAML configs, species presets) are referenced but not publicly linked, and several foundational projects (FORMINDEX, MetaInformAnt, ActiveInferAnts) are cited without DOIs or repository links, limiting independent verification. The Reproducibility Checklist appendix is listed in the table of contents but its contents are not elaborated in the available text, and the abstract's promise of 'open evaluation assets' lacks a concrete DOI or URL.

Positive Findings

- Public GitHub repository 'ant_stack' confirmed accessible with `antstack_core` Python package, CLI entrypoints, and a verified test suite reporting 676 passed tests
- Manifest-tracked artifact policy with SHA-256 checksums and per-run provenance files under `outputs/<run_id>/provenance/` is specified and operationalized
- Physics simulation parameters are explicitly stated as design targets (9GB $\times 2$ physics step, 100 Hz control loop, sensor noise $< 2 \times 10^{-3}$, $\sim 250k$ neurons/agent)
- Dependency management is handled via `uv` (Python) and `bun` (Node), with a `pyproject.toml`-style project structure enabling environment reconstruction
- Virtual Fly Brain (VFB) is cited as a concrete, named neural circuit template resource grounding the AntBrain design
- MuJoCo and PyBullet are named as physics engines; Brian2, Nengo, and SpikingJelly are named as spiking neural simulation backends, enabling partial replication of the simulation stack
- Species-parameterized YAML/JSON manifest system for one-click reruns is described in both the README and the paper's roadmap
- Example config file (`configs/run_all_antstack.example.yaml`) is referenced in the README with a concrete regeneration command
- The paper includes a Reproducibility Checklist appendix (Appendix A) and a Species Parameterization Quickstart (Appendix B), indicating awareness of reproducibility norms
- Wehner (2003) and Seelig & Jayaraman (2015) are cited as specific literature sources for polarized-light navigation and orientation integration parameters

Gaps Identified

- No external dataset accessions are provided: field datasets, arena tracking data, and tools like DeepLabCut and idtracker.ai are mentioned as enabling parameterization and validation but no specific dataset DOIs or repository links are given
- Foundational projects FORMINDEX, MetaInformAnt, and ActiveInferAnts are described as 'core inspirations' with no repository URLs, DOIs, or version pins
- The abstract promises 'open evaluation assets and baseline species configurations' but provides no DOI, URL, or Zenodo deposit for these assets
- The Reproducibility Checklist appendix (Appendix A) is listed in the table of contents but its substantive contents are not elaborated in the available text
- A referenced `Resources.md` file is cited for core literature and datasets used to template modules but is not publicly linked or described
- The GitHub repository does not appear to have a versioned release tag or archived Zenodo DOI, reducing long-term citability of the code artifact
- Species parameter presets (YAML/JSON) are described as planned ('Ship species parameter presets') in the roadmap rather than confirmed as currently available artifacts
- No software license is explicitly mentioned in the available evidence, leaving reuse rights ambiguous
- Calibration of neural circuit parameters to specific ant species is described in prose but no parameter table with cited empirical sources is provided in the paper
- [Critic] No empirical validation data (e.g., locomotion performance metrics, stability benchmarks, or neural activity traces) are cited to confirm that the AntBody simulation actually runs at the stated parameters rather than merely targeting them.
- [Critic] The biological plausibility of the $\sim 250k$ neuron count is asserted without a direct citation to a source establishing this as the

accepted Drosophila neuron count or as computationally tractable at the stated timestep.

- [Critic] No provenance is provided for the specific sensor noise range <2 , ³ ã Å ã UÖ r —B —2 Væ6/ÆV " v†WF†W" F†—2 —2 FW ived from biological measurements, prior robotics literature, or is an arbitrary engineering choice.

Recommendations

- Deposit the current repository state to Zenodo with a DOI and include the DOI in the paper and README to establish a citable, archived code artifact
 - Add a explicit software license (e.g., MIT or Apache 2.0) to the repository
 - Provide concrete repository URLs or DOIs for FORMINDEX, MetaInformAnt, and ActiveInferAnts in the paper's reference list or footnotes
 - Elaborate the contents of Appendix A (Reproducibility Checklist) with a filled-out checklist rather than a placeholder heading
 - Identify at least one publicly accessible field or arena dataset (with accession number and version) used for parameter calibration or validation, even if the full dataset integration is future work
 - Publish the species parameter presets (YAML/JSON) and example experiment manifests in the repository now, even in draft form, so readers can attempt one-click reruns
 - Add a pinned release tag (e.g., v0.1.0) to the GitHub repository corresponding to the paper submission
 - Include a parameter provenance table mapping each key simulation constant (9GBÂ 6öçG&öÂ ate, neuron count, noise <2' Fò —G2 7 V6—f—2 V× — ical or theoretical source citation
 - Provide a CI badge (GitHub Actions or equivalent) in the README to make the test-suite status independently verifiable
 - Create a Zenodo or OSF deposit for the evaluation assets and baseline species configurations referenced in the abstract, with a DOI included in the paper
-

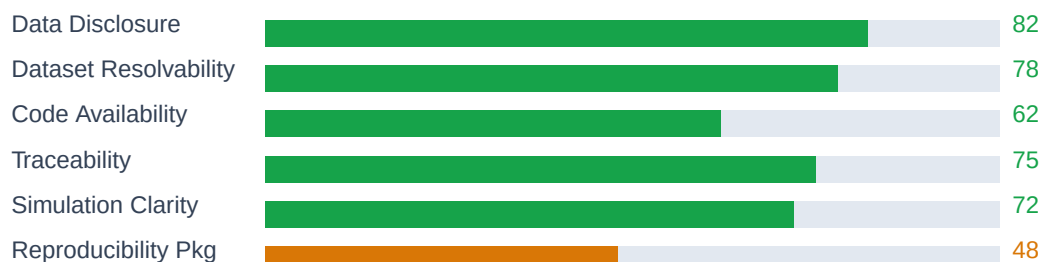
Termite Colony — Unity 3D Multiagent

Source: <https://github.com/Dougarasu/termite-multiagent-system>

Evaluated: June 3, 2026

71.6

Overall /100



Summary

This is a simple agent-based simulation project hosted on GitHub with an MIT license and a basic README describing the simulation concept, agent rules, and reported experimental setup (10 termites, 1000 logs, spawn at origin, 1-second direction change interval). However, the repository has no versioned releases, no dependency manifest, no CI/tests, and was last updated in 2016. Key reproducibility gaps include: missing numeric values for physics parameters (mass, angular/linear speed), unspecified environment dimensions, a mismatched clone URL in the README, and no archived snapshot (e.g., Zenodo DOI). As a purely synthetic simulation with no external datasets, scoring reflects how completely generative inputs and configuration are specified, which is only partially achieved through prose description without config files or parameter tables.

Positive Findings

- Repository is publicly accessible on GitHub under MIT license with license file present
- README documents the two core agent behavioral rules completely and unambiguously
- Experimental setup for reported results is partially specified: 10 termites, 1000 logs, spawn point (0,0,0), 1-second direction change interval
- Agent structure is described at component level (Body, Motion Actuator, Wall Sensor, Log Sensor)
- Environment structure (Scenario, Termite Generator, Log Generator) is described with configurable parameters noted
- Physics attributes used (position, rotation, angular speed, linear speed, mass) are enumerated
- Result progression images at 0/10/30/60 minutes are referenced, demonstrating qualitative validation
- Unity 5.x framework and C# language are specified, enabling environment reconstruction in principle
- Repository has 38 commits providing development history

Gaps Identified

- Clone URL in README (<https://github.com/Dougarasu/TrabalhoIA.git>) does not match the actual repository URL, breaking the installation instructions
- No numeric values provided for any rigidbody physics parameters (mass, angular speed, linear speed)
- No environment dimensions or scale specified, making spatial replication impossible
- No tagged release or pinned commit corresponding to the reported simulation results
- No dependency manifest (e.g., packages.json, .csproj with pinned Unity package versions) present
- No automated tests or CI pipeline detected
- Default values for Termite Generator and Log Generator (spawn delay, max spawn amount) are not documented
- No random seed documented or exposed, making exact result reproduction non-deterministic
- Result images are embedded in README with no downloadable supplementary data or raw output files
- Repository has been inactive since 2016-10-22 with no releases published
- No archived snapshot (Zenodo or equivalent) providing a persistent, citable version
- [Critic] The repository has had no activity since 2016-10-22 (117 months ago), has 0 releases and 0 tags, and no CI or automated tests. There is no versioned snapshot (e.g., Zenodo DOI) explicitly cited in the paper or README, meaning the exact code state used to produce the reported results is not pinned and could drift or become inaccessible over time.
- [Critic] Unity 5.x is no longer actively supported and the specific sub-version used is not recorded in any dependency manifest (none detected), making environment reconstruction for reproducibility non-trivial.

Recommendations

- Fix the clone URL in README to match the actual repository URL
- Add a Unity package manifest or document the exact Unity 5.x version (minor + patch) required
- Document all numeric parameter values in the README or a dedicated config file: termite speed, angular speed, mass, wall detection radius, log detection radius
- Specify the environment dimensions and coordinate system scale explicitly
- Expose and document the random seed used for the reported experiment to enable deterministic reproduction
- Create a tagged GitHub release (e.g., v1.0) or pinned commit hash corresponding to the reported results
- Archive the repository to Zenodo and include the resulting DOI in the README
- Add a configuration file (e.g., JSON or ScriptableObject) capturing all simulation parameters used in reported results as a reproducibility record

- Include raw output data (e.g., log pile counts or positions over time) as downloadable supplementary files rather than only embedded images
 - Document default values for all configurable generator parameters in the README
-

Bee Swarm Simulation

Source: <https://github.com/fractastical/bee-swarm-sim>

Evaluated: June 3, 2026

68.1

Overall /100



Summary

This is a browser-based agent simulation with no external empirical datasets, so evaluation focuses on generative input transparency rather than data accession. The README documents a clear citation-backed mode with 8+ literature references tracing key behavioral parameters (waggle dance direction/distance, follower sampling, thermal balling), and the simulation requires no build step or dependencies, making basic execution straightforward. However, the repository lacks a license, versioned releases, tagged commits, CI, dependency manifests, and an archived DOI snapshot, limiting reproducibility guarantees; crucially, the heuristic mode's hand-tuned parameters are undocumented, and no example trace files, parameter tables, or random seeds are provided to fully reconstruct simulation outputs.

Positive Findings

- Simulation code is publicly accessible at [fractastical/bee-swarm-sim](https://github.com/fractastical/bee-swarm-sim) with a clear README (~5405 bytes) describing all major components
- No build step or external dependencies required; runs directly in a browser from three well-separated files (HTML, CSS, JS)
- Citation-backed mode explicitly maps 8 peer-reviewed references (von Frisch 1967, Seeley 1995, Couvillon 2019, Schurch 2021, Ai 2019, Menzel 2023, Ono 1995, Shannon 1948) to specific behavioral and metric parameters
- README explicitly documents all four core signal primitives (waggle, stop, tremble, shaking) with functional descriptions grounded in literature
- BeeStack trace schema is formally specified with a JSON example, enabling external frame injection and interoperability
- Environment-to-reality mapping is explicitly described with caveats ('scale interpretation tool, not a claim'), showing epistemically responsible framing
- Information-theoretic and circular-statistics metrics are cited to Shannon (1948) and Fisher (1993), providing methodological transparency

Gaps Identified

- No software license detected in repository; code reuse rights are legally ambiguous
- No versioned releases or git tags; no pinned commit referenced, making it impossible to snapshot an exact reproducible state
- No CI pipeline or automated tests; correctness of simulation logic cannot be continuously verified
- Heuristic information model uses 'original hand-tuned approximations' with no documentation of parameter values, derivation, or sensitivity analysis
- No example BeeStack trace files provided despite the schema being specified; external replay mode cannot be tested without manual file construction
- No random seed control documented or exposed; stochastic runs cannot be exactly reproduced
- Parameter tables for agent rules (e.g., foraging thresholds, recruitment probabilities, dance duration distributions) are not provided as config files or supplementary tables
- No archived DOI snapshot (e.g., Zenodo) for long-term citability
- No dependency manifest (package.json or equivalent) even though there are no declared dependencies—this makes the zero-dependency claim unverifiable by tooling
- [Critic] The repository has 0 published releases and 0 tags, meaning there is no versioned snapshot of the code corresponding to any specific paper submission or claimed state; reproducibility of any specific described behavior is therefore contingent on the mutable HEAD commit.
- [Critic] No tests directory or CI pipeline is present, so there is no automated validation that the simulation behaves as described or that citation-backed parameter values are enforced or checked against expected outputs.
- [Critic] The repository has no dependency manifest, which is consistent with the 'no dependencies' claim, but also means there is no formal specification of the execution environment (browser version, JS engine) needed to reproduce results.
- [Critic] The README is ~5405 bytes and contains the citations, but without a structured parameter table or mapping from citation to code constant, the provenance chain from literature to implementation remains asserted rather than documented.

Recommendations

- Add an open-source license (e.g., MIT or Apache 2.0) to clarify reuse rights
- Create a versioned GitHub release or annotated git tag corresponding to any manuscript submission, and archive to Zenodo for a persistent DOI
- Document all hand-tuned heuristic parameters in a structured table (in README or a separate params.json), distinguishing them from citation-backed values

- Expose and document random seed control in the UI and in the export JSON so specific runs can be exactly reproduced
 - Include at least one example BeeStack trace JSON file in the repository so the import/replay feature can be immediately tested
 - Add a GitHub Actions CI workflow that runs a headless smoke test (e.g., using Playwright or Puppeteer) to verify the simulation launches and produces expected metric ranges
 - Provide a parameter sensitivity table or appendix showing how key outputs (e.g., foraging efficiency, dance recruitment rate) change across plausible parameter ranges
 - Add a CITATION.cff or codemeta.json file to standardize how the software itself should be cited
-

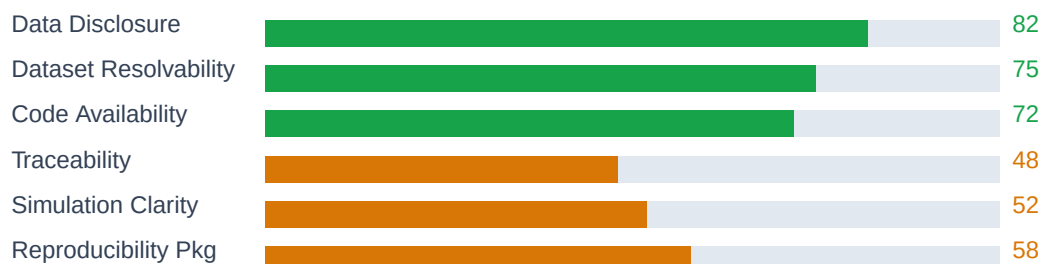
Rust Ant Colony Simulation

Source: <https://github.com/bones-ai/rust-ants-colony-simulation>

Evaluated: June 3, 2026

64.3

Overall /100



Summary

This is a pure agent-based simulation written in Rust using the Bevy game engine, with no external datasets required. The source code is publicly available on GitHub with a Cargo.toml/Cargo.lock dependency manifest and a README providing basic run instructions, supporting a moderate level of reproducibility. However, the repository lacks a software license, tagged releases, CI/CD, tests, and any formal versioning or archive (e.g., Zenodo DOI), and the simulation parameters in src/configs.rs are undocumented with no empirical or theoretical citations tracing their values. The simulation's internal logic (pheromone decay, ant decision rules, kdtree query caching strategy) is not described in any accompanying paper or supplementary documentation, making full behavioral reproducibility difficult.

Positive Findings

- Source code is publicly accessible on GitHub at bones-ai/rust-ants-colony-simulation
- Cargo.toml and Cargo.lock dependency manifests are present, enabling deterministic dependency resolution
- README provides clear clone-and-run instructions (cargo run --release) supporting basic reproducibility
- Configuration file src/configs.rs centralizes simulation parameters, including the documented ANT_INITIAL_PH_STRENGTH parameter
- A demo video (YouTube) and timelapse are referenced, providing qualitative validation of expected behavior
- Internal algorithmic choices (kdtree, query caching) are disclosed in the README
- Being a pure generative simulation, no external dataset accessions are required, and the generative rules are at least partially encoded in the config file

Gaps Identified

- No software license detected in the repository, restricting legal reuse and redistribution
- No tagged releases or pinned commits, making it impossible to pin the exact version used for any reported output
- No Zenodo DOI or other archival snapshot, so the code is not permanently citable
- No CI/CD (GitHub Actions) or automated tests, meaning correctness of the simulation cannot be verified programmatically
- Simulation parameters in src/configs.rs are not documented with empirical sources or theoretical justifications; the specific value of 40.0 for ANT_INITIAL_PH_STRENGTH is unexplained
- No description of the pheromone decay function, ant decision probability model, or random seed handling, making behavioral reproducibility unclear
- No supplementary methods document, preprint, or associated publication explaining the biological or algorithmic basis of the model
- README is minimal (~777 bytes) and does not describe the simulation's rules, state space, or expected outputs
- [Critic] No license is detected in the repository, meaning the terms under which others may reproduce, modify, or redistribute the code are unspecified — relevant if the paper implies open reuse.
- [Critic] No CI/CD workflows are present, so there is no automated evidence that the build instructions ('cargo run --release') continue to produce a working binary on any platform.
- [Critic] No tests directory is detected, meaning correctness of the simulation logic (e.g., kdtree usage, query caching behaviour) is unverified by the repository itself.

Recommendations

- Add an open-source license (e.g., MIT or Apache 2.0) to the repository to permit legal reuse
- Create a tagged GitHub release and deposit the code on Zenodo to obtain a citable DOI
- Document all parameters in src/configs.rs with descriptions, units, default values, and references to ant colony optimization literature (e.g., Dorigo et al.) where applicable
- Add a detailed README or supplementary document describing the pheromone model (evaporation rate, diffusion, deposition rules), ant decision logic (probability functions), and any random seed settings
- Implement at least basic GitHub Actions CI to verify the code compiles and runs without error on each commit
- Specify the random seed or provide a mechanism to fix it, enabling exact reproduction of simulation trajectories
- Add a CHANGELOG or commit-annotated version history so users can track behavioral changes across the 14 commits
- Consider adding a short reproducibility script or example that runs the simulation for a fixed number of steps and reports a summary statistic (e.g., trail convergence metric) for validation

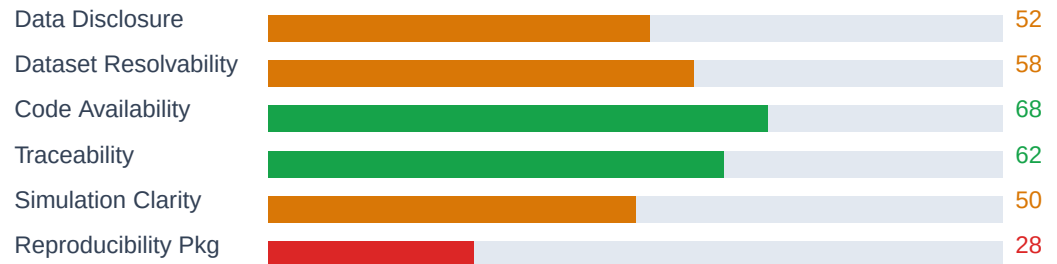
ACO Robot Path Planning (Python)

Source: <https://github.com/Haghran/ACO---Robot-Path-Planning>

Evaluated: June 3, 2026

54.5

Overall /100



Summary

This repository provides a single-file Python simulation of an ACO-based robot path planning algorithm (Liu et al., 2017) and is publicly accessible under GPL-3.0. However, the repository is severely minimal: a ~271-byte README, no dependency manifest, no tests, no CI, no tagged releases, no versioned snapshot (Zenodo/DOI), and no supplementary outputs or configuration files are present. While the simulation uses synthetic environments rather than real-world datasets (appropriate for the domain), the generative parameters, random seeds, and algorithmic constants are not independently documented beyond deferring to the cited reference, which itself lacks a DOI or direct link in the README. The codebase has been stagnant since November 2017 (over 8 years) with only 6 commits, making practical reproducibility dependent entirely on external knowledge of the source paper.

Positive Findings

- Code is publicly available on GitHub under the GPL-3.0 open-source license
- The implementation is explicitly linked to a specific peer-reviewed reference (Liu et al., Soft Computing, 2017)
- The simulation uses synthetic/generated environments, so the absence of external datasets is appropriate and not a defect
- Repository is fully public and accessible with no access restrictions confirmed via live GitHub API signals
- No open issues or unresolved pull requests indicate the codebase has not introduced known bugs reported by users

Gaps Identified

- README is extremely sparse (~271 bytes) and documents neither installation instructions, usage examples, nor parameter descriptions
- No dependency manifest (requirements.txt, setup.py, environment.yml, or equivalent) is present, making environment reconstruction manual
- No tagged release or pinned commit is provided, so the exact code state cannot be unambiguously referenced
- No archived snapshot with a persistent DOI (e.g., Zenodo) exists for the repository
- The cited reference paper (Liu et al., 2017) lacks a DOI or direct URL in the README, reducing traceability
- Simulation parameters, ACO hyperparameters (pheromone evaporation rate, alpha, beta, number of ants, iterations), and random seeds are not documented anywhere in the repository outside the single Python source file
- No supplementary outputs, result logs, or example plots are included to verify expected behavior
- No tests or CI workflow are configured, so correctness cannot be automatically verified
- Repository has not been updated since November 2017 (104 months), raising concerns about compatibility with current Python/library versions
- No data availability statement is present anywhere in the repository

Recommendations

- Expand the README to include: installation instructions, a list of dependencies with version numbers, usage examples with expected outputs, and a description of all key algorithmic parameters with their values
- Add a requirements.txt or conda environment.yml with pinned dependency versions to enable one-command environment reconstruction
- Create a tagged release (e.g., v1.0.0) or reference a specific pinned commit SHA to allow unambiguous citation of the exact code version
- Archive the repository on Zenodo or a similar platform to obtain a persistent DOI and ensure long-term accessibility
- Add a DOI or direct URL for the Liu et al. (2017) reference in the README to improve citation traceability
- Document all ACO hyperparameters (alpha, beta, evaporation rate, number of ants, iteration count, grid dimensions, obstacle configurations, random seeds) explicitly in the README or a separate configuration file
- Include at least one example output plot or result log so users can verify their run matches expected behavior
- Add a minimal test script or assertion checks within PathPlanning.py to validate key algorithmic outputs
- Consider adding a GitHub Actions CI workflow to automatically run the simulation and verify it completes without errors on each commit

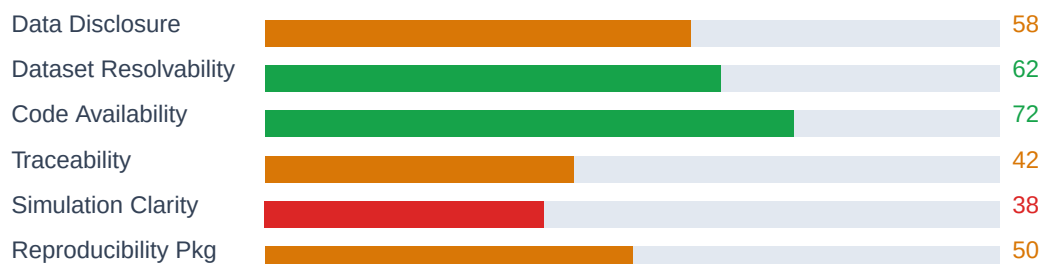
Ant Colony Optimization — React/Canvas

Source: <https://github.com/cfrBernard/ant-colony-optimization>

Evaluated: June 3, 2026

52.7

Overall /100



Summary

This is a React/HTML5 canvas ant-colony simulation learning project with a publicly accessible MIT-licensed GitHub repository containing all bundled assets (sprite sheets, tileset, JSON map). The code and assets are self-contained with no external datasets, and a package.json/package-lock.json provide dependency management. However, no versioned release or commit hash is pinned, no CI or tests are present, simulation parameters and behavior logic are hard-coded without documentation or cited sources, and pathfinding (A*) is explicitly absent, limiting scientific reproducibility. The installation instructions are functional but contain a placeholder username in the clone URL, and no archived snapshot (e.g., Zenodo DOI) exists.

Positive Findings

- Repository is publicly accessible under MIT license at <https://github.com/cfrBernard/ant-colony-optimization>
- All simulation assets (AntMap.json, ants.png sprite sheet, MainLev2.0.png tileset) are bundled directly within the repository, making them immediately available
- Dependency management is handled via npm with both package.json and package-lock.json present, enabling deterministic dependency installation
- Clear installation and execution instructions are provided, including prerequisites (Node.js/npm) and steps to launch at localhost:3000
- README is present (~2354 bytes) with a coherent code structure description mapping files to their roles (Ant.js, AntRenderer.js, MapRenderer.js, AntSimulation.js)
- 14 commit history is present, providing some development traceability
- The project honestly documents its limitations (no A*, no advanced AI), aiding expectation-setting

Gaps Identified

- No published release tags or pinned commit hash referenced for reproducibility; any future commit could silently alter behavior
- No CI/CD pipeline (GitHub Actions or equivalent) detected, so correctness of the build is unvalidated automatically
- No tests directory detected; simulation behavior correctness is entirely unverified
- Simulation parameters (ant speed, state transition thresholds, movement logic) are hard-coded in source files with no documentation, cited sources, or parameter tables
- Clone instruction in README uses a placeholder username (`<your-username>`) rather than the actual repository owner `cfrBernard`, reducing clarity
- No archived snapshot (Zenodo DOI or equivalent) exists, meaning the exact code state is not persistently citable
- Random seed(s) for simulation initialization, if any, are not documented or exposed as configurable parameters
- No Dockerfile or containerized environment provided; local environment variability (Node.js version) could affect reproducibility
- No example output or screenshots included to verify a successful run visually

Recommendations

- Create a versioned GitHub release (e.g., v0.1.0) and/or tag the current stable commit so users can pin to a reproducible state
- Archive the repository on Zenodo to obtain a persistent DOI for citable reference
- Fix the README clone URL to use the actual repository owner (`cfrBernard`) instead of the placeholder `<your-username>`
- Document all simulation parameters (ant speed, state durations, grid dimensions, movement probabilities) in a dedicated config file or parameter table in the README
- Expose any random seeds as configurable constants and document their default values to allow exact reproduction of simulation runs
- Add a `.`npmrc` or `engines` field in package.json specifying the tested Node.js version to reduce environment variability
- Add at minimum a basic smoke test (e.g., that the app builds without errors) and a GitHub Actions workflow to run it on each push
- Include a screenshot or animated GIF of a running simulation in the README so users can confirm their local run matches the expected output

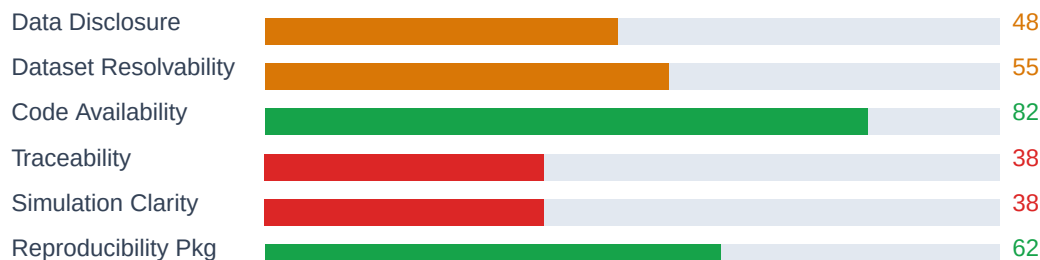
Locas Ants — Lua/Löve2D Ant Colony

Source: <https://github.com/piXelicio/locas-ants>

Evaluated: June 3, 2026

51.6

Overall /100



Summary

This repository is an ant colony simulation remake in Lua/Löve2D with MIT licensing, 6 tagged releases, and a pre-built binary for immediate execution — all positive reproducibility signals. However, as a simulation paper it critically lacks disclosure of the generative parameters, rules, and initial conditions that define agent behavior, and the undocumented Python component (7.7% of code) represents an opaque preprocessing or analysis step. No dependency manifest with pinned versions, no test suite, no CI pipeline, and no archival DOI (e.g., Zenodo) are present, limiting long-term reproducibility confidence. The original simulation it remakes is neither linked nor archived, breaking the provenance chain.

Positive Findings

- Source code is publicly accessible on GitHub under an MIT license, confirmed via live API signals
- 6 versioned releases exist with tags, and a pre-built .love binary is available for one-click execution
- README provides clear step-by-step instructions for both running the binary and working with the source
- Simulation is implemented in a well-known open-source engine (Löve2D) that is freely available and documented
- Repository has 161 commits demonstrating active development history
- Screenshots of simulation output are included in the repository for visual reference
- No external datasets are required, which is appropriate for a pure agent-based simulation

Gaps Identified

- Simulation parameters (pheromone decay rates, ant count, evaporation constants, sensing radii, etc.) are hard-coded in source files with no accompanying parameter table or configuration file documentation
- The Python component (7.7% of codebase) is entirely undocumented in the README — its role, inputs, and outputs are unknown
- No dependency manifest (e.g., lockfile, requirements.txt, or explicit version pins for Lua libraries) is present
- No test suite or CI/CD pipeline exists to verify simulation correctness across environments
- The prior ant colony simulation that this is a remake of is not linked or archived, breaking provenance
- No archival snapshot with a persistent DOI (e.g., Zenodo) exists for long-term citability
- No random seed documentation or initialization conditions are specified, preventing exact reproduction of simulation runs
- The undocumented Python scripts could represent critical data processing or analysis steps with no traceability

Recommendations

- Add a configuration file (e.g., config.lua or params.lua) externalizing all key simulation parameters (pheromone decay, ant population, movement rules) with inline comments explaining each value's biological or design rationale
- Document the Python component explicitly in the README, describing its purpose, inputs, outputs, and how to run it
- Archive the repository on Zenodo to obtain a persistent DOI, enabling formal citation and long-term access
- Add a dependency manifest specifying the exact Löve2D version required, and optionally a Dockerfile or shell script that installs dependencies automatically
- Document the random seed and initial conditions for the simulation so that specific runs can be exactly reproduced
- Link to or archive the original ant colony simulation that inspired this remake to establish provenance
- Add a minimal test or validation script that checks simulation outputs against expected behavioral properties (e.g., pheromone trail formation within N steps)

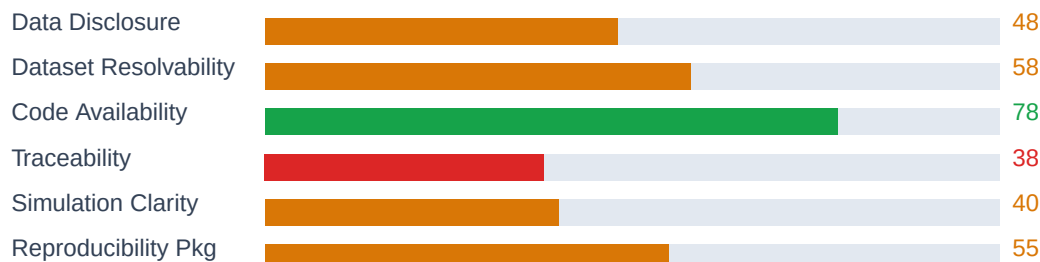
Ants Sandbox (TypeScript/web)

Source: <https://github.com/tulustul/ants-sandbox>

Evaluated: June 3, 2026

51.1

Overall /100



Summary

This repository is a browser-based ant colony simulation with no external datasets, so generative inputs (rules, parameters, initial conditions) serve as the primary disclosure target — yet the README (~490 bytes) documents none of these values. The codebase is publicly available under an MIT license with a dependency manifest (package.json/package-lock.json) and basic build/run instructions, placing it in the mid-range for code availability, but the absence of versioned releases, tags, CI workflows (not detected by the live API scan), tests, or a Zenodo archive prevents a higher score. The inspiration source (a YouTube video) is referenced but not cited with a full URL in the scraped content, and no simulation parameters, pheromone decay rates, ant behavior rules, or random seeds are documented anywhere in the README, making independent validation of the simulation behavior effectively impossible. The project is last-updated July 2023 and lacks containerization or a reproducible environment specification beyond the lockfile.

Positive Findings

- Source code is publicly accessible on GitHub at [tulustul/ants-sandbox](https://github.com/tulustul/ants-sandbox) under an MIT license
- A live deployment is available at <https://ants-sandbox.vercel.app/> for interactive inspection of outputs
- A package-lock.json dependency lockfile is present, providing partial environment reproducibility
- Basic development setup instructions are provided (npm install, npm run start, npm run build, npm run lint)
- A screens/ directory exists providing visual output documentation of expected simulation appearance
- The simulation is a self-contained generative system requiring no external datasets, which is appropriate for the domain
- The codebase uses well-known, auditable technologies (TypeScript 58.8%, Vue 39.1%)

Gaps Identified

- No versioned releases or git tags published (0 releases, 0 tags confirmed by live API), preventing citation of a specific code version
- No archived snapshot with a persistent DOI (e.g., Zenodo) to ensure long-term accessibility
- CI/CD workflows not detected by the live GitHub API scan, contradicting the pre-extracted evidence — automated build verification is absent
- No tests directory detected, meaning simulation correctness cannot be automatically verified
- Simulation parameters (pheromone decay rates, evaporation constants, ant behavior weights, colony size, grid dimensions, time steps) are not documented in the README or any config file surfaced in the scan
- The inspiring YouTube video is not cited with a full URL or reference, making the conceptual basis of the model unverifiable
- No random seed specification or determinism guarantees are documented, preventing exact output reproduction
- No Dockerfile or containerization is provided; environment reproducibility relies solely on the lockfile
- README is extremely sparse (~490 bytes) with no description of simulation mechanics, assumptions, or parameter choices
- No supplementary data files, example outputs with checksums, or validation benchmarks are provided
- [Critic] No published releases or tags exist (0 releases, 0 tags), meaning there is no versioned snapshot of the code tied to any specific paper submission or analysis, making exact reproducibility of a described version difficult.
- [Critic] No tests directory was detected, meaning there is no evidence of automated testing coverage for the codebase.
- [Critic] The repository has not been updated in approximately 35 months (last push 2023-07-15), which may affect whether the live deployment and development setup instructions remain functional with current dependency versions.
- [Critic] The README is minimal (~490 bytes), which limits the documentation of scientific methodology, parameter choices, or interpretation of simulation outputs beyond basic setup commands.

Recommendations

- Publish a versioned GitHub release and/or git tag (e.g., v1.0.0) to enable precise citation of the code state
- Archive the repository on Zenodo to obtain a persistent DOI and ensure long-term accessibility
- Expand the README to document all simulation parameters explicitly: pheromone decay rate, evaporation coefficient, ant sensing radius, colony size, grid dimensions, and any other tunable constants
- Add a full citation (with URL) for the inspiring YouTube video and any other conceptual sources
- Document random seed handling and add an option for deterministic simulation runs to enable exact reproduction
- Add a GitHub Actions CI workflow (reintroduce or confirm the .github/workflows configuration) to automatically verify the build on each commit
- Add a Dockerfile or devcontainer specification to fully pin the runtime environment beyond package-lock.json

- Create at minimum a basic test suite that verifies simulation initialization and a fixed number of steps produce expected outputs
 - Add a configuration file (e.g., JSON or TOML) that externalizes all simulation parameters with inline comments explaining each value's role and valid range
 - Provide example output screenshots with checksums in the screens/ directory so reviewers can validate visual correctness
-

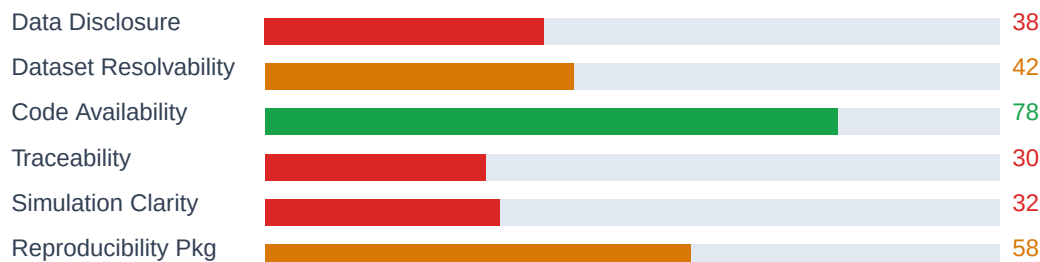
Symbiants — Ant Colony Sim (Rust)

Source: <https://github.com/MeoMix/symbiants>

Evaluated: June 3, 2026

43.8

Overall /100



Summary

Symbiants is an open-source Rust/Bevy ant colony simulation with code publicly available on GitHub (235 stars, 690 commits, Apache-2.0/MIT dual license, Cargo.toml dependency manifest present), but it is not a conventional bioinformatics research paper and lacks virtually all scientific reproducibility infrastructure. The repository has no tagged releases, no CI/CD workflows, no test suite, and no pinned commit for reproducibility. Simulation parameters, behavioral rules, initial conditions, and any generative inputs are entirely undocumented in any citable or structured form, making independent reproduction of simulation behavior practically impossible. There are no external datasets, which is appropriate for a rule-based simulation, but the generative rules themselves are only inferable by reading the source code with no accompanying parameter tables, seeds, or design documents linked from the repository.

Positive Findings

- Source code is publicly accessible on GitHub at MeoMix/symbiants with full commit history (690 commits)
- Repository is dual-licensed under Apache-2.0 and MIT, satisfying open-source license requirements
- A dependency manifest (Cargo.toml and Cargo.lock) is present, providing pinned dependency versions for the Rust ecosystem
- README documents the technology stack (Rust, Bevy game engine, WASM target) and build commands for both native and WASM development
- Development environment is partially reproducible via a VSCode devcontainer (Docker) configuration present in the repository
- Build and run commands are explicitly documented (cargo build/run/watch, trunk build/serve)
- A known platform bug (Android Chrome) is transparently disclosed with reference to the upstream Bevy issue
- External design/lore documentation is linked via Google Docs, providing some narrative context
- No external datasets are used, which is appropriate and not a deficiency for this rule-based simulation

Gaps Identified

- No tagged releases or versioned commits are published (0 published releases, 0 tags confirmed via GitHub API), preventing reproducibility pinning to a specific simulation state
- No CI/GitHub Actions workflows detected, meaning no automated build verification or regression testing exists
- No test directory or test suite detected, so simulation behavior correctness cannot be automatically verified
- Simulation parameters (ant behavior rules, colony growth rates, pheromone decay constants, food spawning rates, initial conditions) are entirely undocumented outside of source code
- No random seeds or stochastic reproducibility controls are documented anywhere in the README or linked documentation
- The Google Docs design document is an external, non-versioned, non-archived resource that could become unavailable or change without notice
- Discord community support is referenced but no link is provided, and no structured issue-resolution documentation exists
- No Zenodo or DOI-based archive of any code snapshot exists
- No example outputs, checksums, or validation benchmarks are provided to confirm a correct simulation run
- Repository has been inactive for approximately 22 months (last push 2024-08-13), raising concerns about maintainability
- [Critic] No CI/CD workflows (GitHub Actions) are detected, meaning there is no automated validation that the documented build commands (cargo build, trunk serve, etc.) actually succeed — reproducibility of the build instructions is unverified.
- [Critic] No tests directory is detected, so the codebase has no evidence of automated correctness validation for any simulation logic described in the project.
- [Critic] Zero published releases and zero tags mean there is no versioned, citable snapshot of the software; any reference to a specific state of the tool cannot be precisely reproduced by a third party.
- [Critic] The Google Docs design/lore document linked in the README is an external, non-archived resource with no DOI or snapshot — its long-term availability and content stability cannot be guaranteed.

Recommendations

- Create a tagged versioned release on GitHub (e.g., v0.1.0-mvp) with a changelog so users can pin to a reproducible code state
- Add a GitHub Actions CI workflow to automatically build for both native and WASM targets on every push, confirming the build remains functional
- Document all key simulation parameters (ant speed, pheromone half-life, food spawn rate, colony growth multipliers, queen behavior thresholds) in a structured table within the README or a dedicated PARAMETERS.md file

- Specify and document the random seed mechanism used in the simulation, or add a configurable seed parameter with documentation
 - Archive a versioned snapshot of the repository on Zenodo to obtain a permanent DOI for citation and long-term preservation
 - Add a minimal test suite (even simple unit tests for core behavioral rules) to enable automated correctness verification
 - Move or mirror the Google Docs design document into a versioned Markdown file within the repository to prevent link rot
 - Provide a Discord invite link in the README and consider adding a structured CONTRIBUTING.md and issue template
 - Add a one-command reproducibility script (e.g., a Makefile or shell script) that sets up the environment and runs a demonstration simulation end-to-end
 - Document the devcontainer Dockerfile more explicitly with version pins for system dependencies to ensure environment reproducibility over time
-

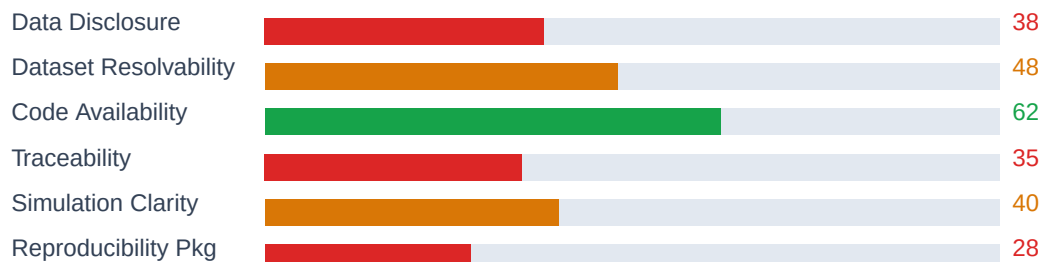
swarm-abc — Artificial Bee Colony Simulation

Source: <https://github.com/darwiiiish/swarm-abc>

Evaluated: June 3, 2026

41.9

Overall /100



Summary

This repository presents a browser-based Artificial Bee Colony simulation implemented in HTML, JavaScript, and CSS, and is publicly accessible on GitHub without authentication. However, the repository is severely underdocumented: it contains no README, no license, no dependency manifest, no CI configuration, no tagged releases, and only a single commit, making independent reproducibility nearly impossible. Simulation parameters (colony size, limit, number of food sources, iteration counts) are not disclosed in any configuration file or documentation, and no foundational ABC algorithm reference is cited anywhere in the repository metadata or description. While the simulation legitimately requires no external datasets, the generative inputs and algorithmic rules are not traceable from publicly available sources, leaving the scientific basis of the implementation unverifiable.

Positive Findings

- Repository is publicly accessible on GitHub without authentication, confirmed via live API signals
- Source code implementing the ABC algorithm is present (abc.js, JavaScript, 37.7% of codebase)
- An explanation document (explanation.html) is included, suggesting some intent to document the simulation
- Repository was recently pushed (2026-05-23), indicating the code is not abandoned
- As a rule-based simulation, no external datasets are required, which is appropriate for this class of model

Gaps Identified

- No README file is present in the repository, leaving setup, purpose, and usage entirely undocumented
- No license is declared, making legal reuse and redistribution ambiguous
- No dependency manifest (e.g., package.json) is present, so the runtime environment cannot be reliably reconstructed
- No simulation parameters (colony size, limit, food sources, iteration count, random seeds) are documented in any configuration file or prose
- No foundational ABC algorithm paper or methodological source is cited in the repository description or any documentation file
- No tagged release or pinned commit is provided, so there is no versioned snapshot of the code
- No CI or automated testing workflows are configured, so correctness of the implementation cannot be continuously validated
- No archived snapshot (e.g., Zenodo DOI) exists for long-term citability
- The repository has zero community engagement (0 stars, 0 forks, 0 open issues), indicating no external validation
- Only a single commit exists, suggesting no iterative development history that could help trace algorithmic decisions
- [Critic] The repository has no license detected, meaning the code's reusability and redistribution rights are legally undefined — a relevant provenance concern for claimed public availability.
- [Critic] No README is present, so there is no documented entry point, usage instructions, or description confirming the repository's purpose beyond filename inference.
- [Critic] The repository has 0 published releases and 0 tags, meaning there is no versioned snapshot corresponding to what was used in the paper, making exact reproducibility of the reported results unattainable.
- [Critic] No tests directory or CI configuration is detected, so there is no evidence the implementation has been validated for correctness.

Recommendations

- Add a detailed README.md covering: project purpose, ABC algorithm variant implemented, all simulation parameters with default values, instructions to run locally, and a reference to the foundational Karaboga (2005) ABC paper
- Declare all simulation parameters (colony size, number of food sources, limit, max cycles, random seed) in a separate JSON or YAML configuration file to enable systematic reproducibility
- Add an open-source license (e.g., MIT or Apache 2.0) to clarify reuse rights
- Create a tagged release (e.g., v1.0.0) or at minimum document the specific commit hash used for any associated publication
- Archive the repository on Zenodo to obtain a persistent DOI for citation purposes
- Add a package.json or equivalent manifest if any build/test tooling is used, even for a browser-only project
- Implement at least basic automated tests (e.g., unit tests for the core ABC update functions) and configure GitHub Actions CI to run them
- Cite the original ABC algorithm source (Karaboga, 2005, or equivalent) in the repository About section, explanation.html, or README
- Document the random seed strategy or provide a seeded mode so simulation runs can be exactly reproduced

Hardware-Accelerated Ant Colony Swarm (C++)

Source: <https://github.com/matheuslosilva/Hardware-Accelerated-Ant-Colony-Based-Swarm-System>

Evaluated: June 3, 2026

37.8

Overall /100

Data Disclosure		32
Dataset Resolvability		38
Code Availability		58
Traceability		35
Simulation Clarity		30
Reproducibility Pkg		40

Summary

This repository provides a publicly accessible simulation of a hardware-accelerated ant colony swarm system, with source code on GitHub and a minimal README containing build instructions. However, the repository lacks a license, versioned releases or pinned commit, dependency manifests with pinned versions, CI/CD, tests, and any documentation of simulation parameters, evolutionary algorithm configuration, or random seeds — making full reproducibility extremely difficult. As a simulation paper with no external datasets, the critical reproducibility burden falls on disclosing generative inputs (rules, parameters, initial conditions), none of which are documented. The README (~1264 bytes) provides only installation steps and high-level description, offering no parameter tables, no result datasets, and no container or environment specification.

Positive Findings

- Source code is publicly accessible on GitHub with 38 commits and recent activity (last push 2026-05-16)
- A Makefile is provided enabling straightforward build and execution via 'make' and 'make run'
- Git submodules are used for additional dependency management, indicating some awareness of reproducible dependency tracking
- External dependencies (GLFW, GLM) are named and installation commands are provided for Debian and Arch Linux
- Three OpenGL/CUDA reference implementations are cited in the README, providing some technical provenance for the rendering stack
- The simulation-only design means no external dataset accessions are required, which is appropriate for this problem domain

Gaps Identified

- No software license is present in the repository, preventing legitimate reuse or redistribution
- No versioned release, tag, or pinned commit hash is cited, making it impossible to reproduce a specific version of the codebase
- No archived snapshot (e.g., Zenodo DOI) exists, so the code has no persistent, citable identifier
- Simulation parameters (pheromone evaporation rate, number of ants, search space dimensions, sensor range, etc.) are entirely undocumented
- Evolutionary algorithm configuration (population size, mutation rate, crossover strategy, fitness function, termination criteria) is not disclosed anywhere in the repository
- Random seeds for the simulation and evolutionary algorithm are not specified, preventing deterministic reproduction of results
- No dependency manifest with pinned versions (e.g., requirements file, CMakeLists with version constraints) is provided; CUDA version required is unspecified
- No container or environment specification (Dockerfile, conda environment, Nix flake) is provided, making build environment reconstruction manual
- No CI/CD pipeline exists to verify that the build remains functional
- No test suite is present to validate simulation correctness
- No result datasets, output examples, or benchmark figures are deposited anywhere
- No data availability statement is present in the README or repository documentation
- The specific CUDA compute capability or GPU hardware requirements are not documented
- [Critic] No dependency manifest (e.g., CMakeLists.txt, requirements file, or package.json) is detected at the repo root, meaning the exact versions of GLFW, GLM, and other dependencies used in the reported results are not pinned or reproducibly specified.
- [Critic] The repository has 0 published releases and 0 tags, so there is no versioned snapshot of the code corresponding to any specific paper submission or experimental result.
- [Critic] No test directory or automated tests are detected, meaning correctness of the CUDA/OpenGL simulation logic cannot be independently verified from the repository alone.

Recommendations

- Add a LICENSE file (e.g., MIT or GPL) to enable legal reuse of the code
- Create a versioned GitHub release or at minimum document the specific commit hash used for any published results
- Archive the repository on Zenodo (linked from a GitHub release) to obtain a persistent DOI
- Add a detailed parameter table to the README or a dedicated configuration file documenting all ant colony parameters, evolutionary algorithm hyperparameters, and initial conditions
- Expose all simulation parameters through a configuration file (e.g., JSON/YAML/TOML) rather than hard-coded values, and include an example config in the repository
- Document the required CUDA version, compute capability, and GPU hardware used in experiments

- Add a Dockerfile or conda environment specification that pins CUDA, GLFW, and GLM versions to ensure reproducible build environments
 - Report and fix the fixed random seeds used in experiments, or document the seeding strategy explicitly
 - Add example output data or result figures so users can validate their build produces correct results
 - Implement a minimal test (e.g., a short simulation run with known output) to verify correctness after build
 - Set up GitHub Actions CI to automatically build the project on each commit
-

